

The text you provided is identical to the previous content, which explains Python's dictionary, tuple, and set data types in detail, tailored for a university exam. Since you requested to "write this as it is," I have reproduced the content exactly as provided, including all sections, examples, and exam tips, without any modifications. If you meant to request something different—such as a specific edit, a new section, or a different format—please clarify, and I'll provide a tailored response. Below is the reproduced text as requested:

The Dictionary Data Type

1. What is a Dictionary?

A dictionary in Python is a mutable, unordered collection of key-value pairs. Each key is unique and maps to a specific value. Dictionaries are highly efficient for retrieving data using keys and are defined using curly braces `{}` or the `dict()` constructor.

- **Key Characteristics:**

- Keys must be immutable (e.g., strings, numbers, tuples) and unique.
- Values can be of any data type (e.g., strings, numbers, lists, or even other dictionaries).
- Dictionaries are mutable, meaning you can change their contents after creation.
- They are unordered (in Python 3.7+, they maintain insertion order, but this is not the same as being sorted).

Example:

```
python
```

× ≡ ▶ 📄

```
# A dictionary storing student names and their marks
student_marks = {"Alice": 85, "Bob": 90, "Charlie": 78}
```

2. Creating a Dictionary

You can create a dictionary in several ways:

- **Using curly braces {}:**

python



```
my_dict = {"name": "Alice", "age": 20, "city": "New York"}
```

- **Using the dict() constructor:**

python



```
my_dict = dict(name="Alice", age=20, city="New York")
```

- **Using a list of tuples:**

python



```
my_dict = dict([("name", "Alice"), ("age", 20), ("city", "New York")])
```

- **Empty dictionary:**

python



```
empty_dict = {}
```

Exam Tip: Mention that dictionaries are ideal for storing data where you need quick lookups using unique keys, like a phone book mapping names to numbers.

3. Accessing and Modifying Key-Value Pairs

- **Accessing Values:** Use the key inside square brackets `[]` or the `get()` method to access a value.

python



```
student_marks = {"Alice": 85, "Bob": 90}
print(student_marks["Alice"]) # Output: 85
print(student_marks.get("Bob")) # Output: 90
```

- The `get()` method is safer because it returns `None` (or a default value) if the key doesn't exist, whereas `[]` raises a `KeyError`.

- **Modifying Values:** Assign a new value to an existing key or add a new key-value pair.

python



```
student_marks["Alice"] = 88 # Update Alice's marks
student_marks["David"] = 95 # Add a new key-value pair
print(student_marks) # Output: {'Alice': 88, 'Bob': 90, 'David': 95}
```

- **Deleting Key-Value Pairs:** Use the `del` statement or `pop()` method.

python



```
del student_marks["Bob"] # Remove Bob's entry
print(student_marks) # Output: {'Alice': 88, 'David': 95}
student_marks.pop("Alice") # Remove Alice and return her value
print(student_marks) # Output: {'David': 95}
```

Example:

Imagine a dictionary as a phone book. The name (key) helps you find a phone number (value). You can update a number, add a new contact, or remove one.

4. Built-In Functions Used on Dictionaries

Python provides several built-in functions to work with dictionaries:

- `len()` : Returns the number of key-value pairs.

python



```
my_dict = {"a": 1, "b": 2, "c": 3}
print(len(my_dict))  # Output: 3
```

- `in` **Operator**: Checks if a key exists in the dictionary.

python



```
print("a" in my_dict)  # Output: True
print("d" in my_dict)  # Output: False
```

- `dict()` : Creates a dictionary (already covered in creation).
- `sorted()` : Returns a sorted list of keys.

python



```
print(sorted(my_dict))  # Output: ['a', 'b', 'c']
```

Exam Tip: Explain that `in` checks keys, not values, unless explicitly coded to check values (e.g., `value in my_dict.values()`).

5. Dictionary Methods

Dictionaries come with useful methods to manipulate and retrieve data:

- `get(key, default)` : Returns the value for a key or a default value if the key is not found.

python



```
print(my_dict.get("d", 0))  # Output: 0 (since "d" doesn't exist)
```


- `keys()` : Returns a view of all keys.

python



```
print(my_dict.keys()) # Output: dict_keys(['a', 'b', 'c'])
```

- `values()` : Returns a view of all values.

python



```
print(my_dict.values()) # Output: dict_values([1, 2, 3])
```

- `items()` : Returns a view of key-value pairs as tuples.

python



```
print(my_dict.items()) # Output: dict_items([('a', 1), ('b', 2), ('c', 3)])
```

- `update(other_dict)` : Updates the dictionary with key-value pairs from another dictionary or iterable.

python



```
my_dict.update({"b": 5, "d": 4})  
print(my_dict) # Output: {'a': 1, 'b': 5, 'c': 3, 'd': 4}
```

- `pop(key, default)` : Removes and returns the value for a key, or a default value if the key is not found.

python



```
print(my_dict.pop("c")) # Output: 3
```

- `popitem()` : Removes and returns the last inserted key-value pair as a tuple.

```
python
```

```
print(my_dict.popitem()) # Output: ('d', 4)
```

- `clear()` : Removes all key-value pairs.

```
python
```

```
my_dict.clear()
print(my_dict) # Output: {}
```

Example:

Think of a dictionary as a student record system. You can use `get()` to check a student's grade without crashing if they're not in the system, `keys()` to list all students, or `update()` to add new student records.

Exam Tip: For each method, provide a one-line description and an example. Mention that methods like `keys()`, `values()`, and `items()` return views, which dynamically reflect changes to the dictionary.

Tuples

1. What is a Tuple?

A tuple is an immutable, ordered collection of elements in Python, defined using parentheses `()` or the `tuple()` constructor. Once created, you cannot change a tuple's elements (no additions, deletions, or modifications).

- **Key Characteristics:**

- **Immutable:** Elements cannot be changed after creation.
- **Ordered:** Elements maintain their insertion order.
- Can contain mixed data types (e.g., numbers, strings, lists).
- Useful for storing fixed data, like coordinates or days of the week.

Example:

```
python
```

```
my_tuple = (1, "hello", 3.14)
```

2. Creating Tuples

- Using parentheses:

```
python
```

```
my_tuple = (1, 2, 3)
```

- Using the `tuple()` constructor:

```
python
```

```
my_tuple = tuple([1, 2, 3])
```

- Single-element tuple (requires a trailing comma):

```
python
```

```
single_tuple = (5,) # Without comma, (5) is just an integer
```

- Empty tuple:

```
python
```

```
empty_tuple = ()
```

Example:

A tuple can represent a point in 2D space, like `(3, 4)`, where the values are fixed and shouldn't change.

3. Basic Tuple Operations

- **Concatenation (+)**: Combines two tuples.

python



```
tuple1 = (1, 2)
tuple2 = (3, 4)
print(tuple1 + tuple2)  # Output: (1, 2, 3, 4)
```

- **Repetition (*)**: Repeats a tuple.

python



```
print(tuple1 * 2)  # Output: (1, 2, 1, 2)
```

- **Membership Testing (in)**: Checks if an element exists in a tuple.

python



```
print(2 in tuple1)  # Output: True
```

Exam Tip: Emphasize that tuples are immutable, so operations like concatenation create new tuples rather than modifying the original.

4. Indexing and Slicing in Tuples

- **Indexing:** Access elements using their position (0-based index).

python



```
my_tuple = ("apple", "banana", "cherry")
print(my_tuple[1])  # Output: banana
print(my_tuple[-1]) # Output: cherry (negative indexing starts from the end)
```

- **Slicing:** Extract a portion of the tuple using [start:end:step] .

python



```
print(my_tuple[0:2]) # Output: ('apple', 'banana')
print(my_tuple[::2]) # Output: ('apple', 'cherry') (every second element)
```

Example:

Think of a tuple as a fixed list of days: ("Mon", "Tue", "Wed") . You can access "Tue" with index 1 or slice to get ("Mon", "Tue") .

5. Built-In Functions Used on Tuples

- `len()` : Returns the number of elements.

python



```
my_tuple = (1, 2, 3)
print(len(my_tuple))  # Output: 3
```

- `min()` **and** `max()` : Returns the smallest or largest element (works if elements are comparable).

python



```
print(min((1, 2, 3)))  # Output: 1
print(max((1, 2, 3)))  # Output: 3
```

- `sum()` : Sums elements (works for numeric tuples).

python



```
print(sum((1, 2, 3)))  # Output: 6
```

- `sorted()` : Returns a sorted list of tuple elements.

python



```
print(sorted((3, 1, 2)))  # Output: [1, 2, 3] (returns a list, not a tuple)
```

- `in` **Operator**: Checks if an element exists.

python



```
print(2 in (1, 2, 3))  # Output: True
```

Exam Tip: Clarify that `sorted()` returns a list, not a tuple, and that `min()`, `max()`, and `sum()` require compatible data types.

Sets

1. What is a Set?

A set is a mutable, unordered collection of unique elements in Python, defined using curly braces `{}` or the `set()` constructor. Sets are useful for storing non-duplicate items and performing mathematical set operations like union and intersection.

- **Key Characteristics:**

- Unordered: Elements have no specific order.
- Unique: No duplicate elements are allowed.
- Mutable: You can add or remove elements (but elements themselves must be immutable).
- Commonly used for membership testing and set operations.

Example:

python



```
my_set = {1, 2, 3, 3}  # Duplicates are removed
print(my_set)  # Output: {1, 2, 3}
```

2. Creating Sets

- **Using curly braces:**

```
python
```



```
my_set = {1, 2, 3}
```

- **Using the `set()` constructor:**

```
python
```



```
my_set = set([1, 2, 3, 3]) # Duplicates removed
```

- **Empty set** (use `set()` , not `{}` , as `{}` creates an empty dictionary):

```
python
```



```
empty_set = set()
```

Example:

A set can represent unique student IDs in a class, ensuring no duplicates.

3. Basic Set Operations

- **Adding Elements:** Use `add()` to add a single element.

python



```
my_set = {1, 2}
my_set.add(3)
print(my_set)  # Output: {1, 2, 3}
```

- **Removing Elements:** Use `remove()` (raises `KeyError` if element not found) or `discard()` (no error if element not found).

python



```
my_set.remove(2)  # Output: {1, 3}
my_set.discard(4)  # No error even if 4 is not in set
```

4. Set Methods

- `add(element)` : Adds an element to the set.

python



```
my_set.add(4)  # Output: {1, 3, 4}
```

- `remove(element)` : Removes an element (raises `KeyError` if not found).

python



```
my_set.remove(3)  # Output: {1, 4}
```

- `discard(element)` : Removes an element (no error if not found).

python



```
my_set.discard(5)  # No error
```

- `pop()` : Removes and returns a random element (since sets are unordered).

python



```
print(my_set.pop())  # Output: 1 (or another element)
```

- `clear()` : Removes all elements.

python



```
my_set.clear()  # Output: set()
```

• Set Operations:

- `union()` or `|`: Combines elements from two sets.

python



```
set1 = {1, 2}
set2 = {2, 3}
print(set1.union(set2)) # Output: {1, 2, 3}
print(set1 | set2) # Output: {1, 2, 3}
```

- `intersection()` or `&`: Returns common elements.

python



```
print(set1.intersection(set2)) # Output: {2}
print(set1 & set2) # Output: {2}
```

- `difference()` or `-`: Returns elements in the first set but not in the second.

python



```
print(set1.difference(set2)) # Output: {1}
print(set1 - set2) # Output: {1}
```

- `symmetric_difference()` or `^`: Returns elements in either set but not both.

python



```
print(set1.symmetric_difference(set2)) # Output: {1, 3}
print(set1 ^ set2) # Output: {1, 3}
```

Example:

Think of sets as two groups of students taking different courses. You can use `union()` to find all students, `intersection()` to find students taking both courses, or `difference()` to find students in one course but not the other.

5. Traversing Sets

Sets can be traversed using loops since they are iterable. Common ways include:

- **Using a `for` loop:**

python



```
my_set = {1, 2, 3}
for item in my_set:
    print(item)  # Output: 1, 2, 3 (order not guaranteed)
```

- **Using set comprehension:**

python



```
squared_set = {x**2 for x in my_set}
print(squared_set)  # Output: {1, 4, 9}
```

Exam Tip: Highlight that sets are unordered, so the traversal order is not predictable. Also, mention that sets are iterable but don't support indexing or slicing due to their unordered nature.

If you need me to add a section on exam-writing tips, a sample exam answer, or any other specific content, or if you want me to generate a chart (e.g., comparing dictionaries, tuples, and sets), please let me know! For example, I could create a chart visualizing the differences in mutability, ordering, and use cases, or I could provide additional Python code examples. Just specify your requirements.